# Lesson 10: Open API

## Intro to Programming — Code the Dream

# Session Overview

**By the end of this session, you will be able to:**

- Explain what an open API is and why developers use them
- Read basic API documentation to find an endpoint and a URL
- Explain what an API key is and how to use one safely
- Make a `fetch()` call to a public API and display the response
- Plan a two-endpoint API page that connects to your portfolio

# Explore

# What Is an Open API?

An **API** (Application Programming Interface) lets your code talk to another service.

An **open API** (or public API) is free to use — no payment required.

Examples you may have heard of:

- A weather app gets its data from a weather API
- A map uses a maps API
- This week, **you** will fetch real data and show it on a page

> APIs send data back as JSON — the same format you've already seen with `fetch()`.

# API Options This Week

Pick one that interests you:

| API | What it has |
|---|---|
| Open-Meteo | Weather data |
| Swapi.Tech | Star Wars characters and films |
| ARTIC | Artwork from the Art Institute of Chicago |
| TheDogAPI / TheCatAPI | Dog and cat images |
| Marvel | Marvel characters and comics |
| API-Sports (Soccer) | Football stats |

> Each API has documentation — a guide that shows you what data is available and how to ask for it.

# Reading API Documentation

Before writing any code, read the docs. Look for:

1. **Base URL** — where all requests start (e.g. `https://api.artic.edu/api/v1`)
2. **Endpoints** — the specific paths that return data (e.g. `/artworks`)
3. **Parameters** — options you can add to filter or limit results
4. **Response format** — what fields come back in the JSON

> TheDogAPI and TheCatAPI include ready-to-use fetch examples in their docs — great starting points.

# Making a Fetch Call

The pattern is the same as Lesson 9:

```
fetch("https://api.artic.edu/api/v1/artworks?limit=3")
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.log("Error:", error));
```

**Try this in your browser console right now.**

Open the browser console:

- **Chrome / Edge:** Right-click → Inspect → Console
- **Firefox:** Right-click → Inspect → Console

# Reading the Response

API responses are objects. You have to dig in to find the data you want.

```
fetch("https://api.artic.edu/api/v1/artworks?limit=3")
  .then(response => response.json())
  .then(data => {
    let artworks = data.data;       // the array of results
    let title = artworks[0].title; // first artwork's title
    console.log(title);
  });
```

> If you see `[object Object]` on the page, that means you need to go deeper into the object.

# Check for Understanding

**An API returns this JSON. How do you access the dog's name?**

```
{
  "dog": {
    "name": "Biscuit",
    "breed": "Labrador"
  }
}
```

A) `data.name`

B) `data.dog.name`

C) `data["Biscuit"]`

D) `data[0].name`

# What Is an API Key?

Some APIs require a **key** — a unique code that identifies your app.

Think of it like a library card: the library uses it to track who is borrowing books.

```
const API_KEY = "your_key_here";

let url = `https://api.example.com/data?api_key=${API_KEY}`;

fetch(url)
  .then(response => response.json())
  .then(data => console.log(data));
```

> For this class, storing the key as a `const` in your JS file is fine.

# What Happens Without a Key?

If a key is required and you forget it, the server sends back an error — not data.

Common responses:

- **401 Unauthorized** — no key provided
- **403 Forbidden** — key is wrong or expired

```
fetch("https://api.example.com/data") // no key!
  .then(response => {
    console.log(response.status); // 401
    return response.json();
  })
  .then(data => console.log(data)); // error message
```

> Always check `response.status` if you are not getting data back.

# Check for Understanding

**You make a fetch call and get a `401` status code. What does that mean?**

A) The request worked — `401` is the number of results

B) The server is down and you should try again later

C) The server rejected the request because no valid API key was included

D) Your internet connection is too slow

# Two Endpoints Required

Your assignment requires **two separate fetch calls** to two different endpoints.

**Example with Open-Meteo:**

- **Call 1:** Get the current temperature
- **Call 2:** Get the wind speed

Each button or nav link in your HTML should trigger a **different** fetch.

> Do not fetch everything at once and hide parts of it. Make two real API calls.

# Check for Understanding

**A student writes one fetch call and uses JavaScript to show/hide different parts of the response. Does this meet the two-endpoint requirement?**

A) Yes — the user sees two different views

B) Yes — one fetch call can return all the data needed

C) No — two endpoints means two separate fetch calls to two different API URLs

D) It depends on which API they are using

# Apply

# Core Activity: Let's Build an API Page

We will build a minimal Open API page step by step — **tell me what to type!**

We will use the **ARTIC API** (Art Institute of Chicago) — no key needed.

**Goal:** Two buttons, two fetch calls, results shown on the page.

> This is the same structure you will use in your assignment.

# Step 1 — The HTML Shell

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Art Explorer</title>
  <link rel="stylesheet" href="css/openapi.css">
</head>
<body>
  <nav>
    <a href="index.html">Portfolio</a>
    <button id="btn-artworks">Browse Artworks</button>
    <button id="btn-artists">Browse Artists</button>
  </nav>
  <main id="display"></main>
  <script src="js/openapi.js"></script>
</body>
</html>
```

# Step 2 — First Fetch Call

```
const display = document.getElementById("display");

function fetchArtworks() {
  fetch("https://api.artic.edu/api/v1/artworks?limit=5")
    .then(response => response.json())
    .then(data => {
      let artworks = data.data;
      display.innerHTML = "";
      artworks.forEach(art => {
        display.innerHTML += `<p>${art.title}</p>`;
      });
    })
    .catch(error => {
      display.innerHTML = "<p>Something went wrong.</p>";
    });
}
```

# Step 3 — Second Fetch Call

```
function fetchArtists() {
  fetch("https://api.artic.edu/api/v1/agents?limit=5")
    .then(response => response.json())
    .then(data => {
      let artists = data.data;
      display.innerHTML = "";
      artists.forEach(artist => {
        display.innerHTML += `<p>${artist.title}</p>`;
      });
    })
    .catch(error => {
      display.innerHTML = "<p>Something went wrong.</p>";
    });
}
```

This is the **second endpoint** — a different URL, a different fetch, different data.

# Step 4 — Connect the Buttons

```
document
  .getElementById("btn-artworks")
  .addEventListener("click", fetchArtworks);

document
  .getElementById("btn-artists")
  .addEventListener("click", fetchArtists);
```

**Test it:** Click each button. Do you see different data each time?

# Extension Activity

*Only if students are ready and time allows.*

# Extension — Display Rich Data

Instead of plain text, display a card with multiple fields.

```
artworks.forEach(art => {
  display.innerHTML += `
    <div class="card">
      <h2>${art.title}</h2>
      <p>Artist: ${art.artist_title || "Unknown"}</p>
      <p>Year: ${art.date_display || "N/A"}</p>
    </div>
  `;
});
```

**Challenge:** Add an image using the ARTIC image URL format:

`https://www.artic.edu/iiif/2/{image_id}/full/400,/0/default.jpg`

# Assignment Preview

This week you will build your **Open API page**:

- [ ] New `openapi.html`, `openapi.css`, and `openapi.js` files
- [ ] A nav link from your portfolio (`index.html`) to the new page
- [ ] A nav link on the new page back to your portfolio
- [ ] Two separate `fetch()` calls to two different endpoints
- [ ] Both results displayed on the page
- [ ] Error handling with `.catch()`
- [ ] Review the **Open API Rubric** before submitting

# Tips for the Assignment

1. **Choose your API first** — read the docs before writing any code
2. **Test in the browser console** — paste a fetch call and explore the response
3. **Log the whole response first** — `console.log(data)` before accessing properties
4. **If you see** `[object Object]` — you need to go one level deeper
5. **Stuck?** Use the AI prompts from the lesson materials to get hints without getting answers

> "Can you ask me 3 questions that will help me figure out how to access the specific properties inside the returned object?"

# Wrap-Up

**Zoom chat:** Rate your confidence from 1 to 5

> How comfortable do you feel making a fetch call and reading the response?

## Before next session:

- **Choose your API** and read its documentation
- **Build your Open API page** (HTML, CSS, JS)
- Fetch from **two endpoints** and display both results
- Add **error handling** with `.catch()`
- Review the **Open API Rubric**
- Post questions in the **Slack** `#discussion` channel anytime

# Resources

| Resource | What it's for |
|---|---|
| Open-Meteo | Free weather API — no key needed |
| ARTIC API | Free art data API — no key needed |
| TheDogAPI / TheCatAPI | Free — includes fetch examples in docs |
| Swapi.Tech | Free Star Wars data API |
| Marvel API | Requires a free API key |
| MDN: Using Fetch | Reference for the fetch pattern |
| Open API Rubric | Linked in your lesson materials |

> **Next week:** Final Project — putting it all together!

# Great work today!

**You're building something entirely your own.**

Questions? Reach out on Slack anytime.